جامعة الحسن الثاني بالدار البيضاء
UNIVERSITÉ HASSAN II DE CASABLANCA

FSBM
FACULTE DES SCIENCES BEN M'SICK
UNIVERSITE HASSAN II DE CASABLANCA

MODULE :

Le Cloud Computing & DevOps

Pr. F. Benabbou
Master DSBD & IA
2025-2026
Faculté des Sciences Ben M'Sick Casablanca

TABLE OF CONTENTS

01 CLOUD COMPUTING

- Introduction générale
- La Virtualisation
- Les concepts de base du Cloud Computing
- Technologies émergentes du CC : Edge, Fog, …
- Étude de cas et projet pratique

02 DevOps & Cloud

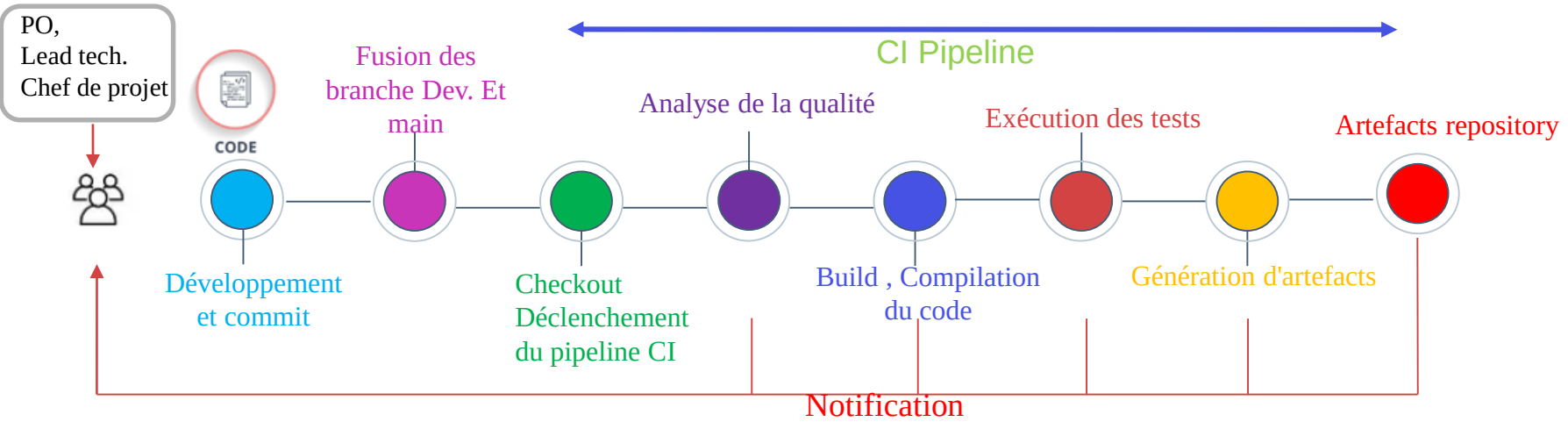
- La philosophie DeVops
- Version control systems (git)
- Intégration Continue CI
- Déploiement Continu CD
- Tests automatisés
- Infrastructure en tant que Code (IaC)
- Surveillance et Journalisation
- Étude de cas et projet pratique

02

CI/CD/CT

Continue Integration (CI)

- L'intégration continue est une pratique de développement qui permet de produire une nouvelle version du logiciel.
- L'intégration continue commence au moment où le développeur envoie une nouvelle version dans le dépôt partagé (commit/push).
- Le CI se termine lorsque le code est validé et qu'un artefact prêt est généré.



Etapes du pipeline CI

- **Développement et commit** : Les développeurs travaillent sur leurs fonctionnalités respectives; une fois une tâche est terminée, le développeur effectue un **commit** de ses modifications dans le dépôt de code source (**Git**, **SVN**, etc.).
- **Déclenchement automatique du pipeline CI** : Chaque commit doit **déclencher** automatiquement l'exécution du **pipeline CI** qui consiste en une série d'étapes automatisées qui vont vérifier et valider les modifications.

Etapes du processus CI : analyse de la qualité

- **Analyse de la qualité du code** : cette phase a pour objectif **d'inspecter** le code et ses composants **sans** les **exécuter**, afin d'évaluer sa qualité, sa sécurité, sa conformité décider si la version sera déployée ou non.
- La phase d'analyse a 4 rôles fondamentaux :
  - ✓ **Qualité** :
    - Détecter les mauvaises pratiques (**Code Smells**), Duplication de code, Code peu lisible,
    - Documentation,
    - Analyse de la Couverture de Test: vérifie la présence des fichiers de tests,
    - Mesurer la complexité cyclomatique : mesure le nombre de chemins linéaires indépendants à travers le code source.

Etapes du processus CI: analyse de la qualité

- Les 4 rôles fondamentaux de la phase d'analyse:
  - ✓ **Sécurité** : Détecter les vulnérabilités
    - SAST (Static Application Security Testing): Recherche de failles dans le code source comme injections SQL, XSS, etc.
      - SonarQube, Fortify, Checkmarx, Veracode
    - SCA (Software Composition Analysis) : Identification des vulnérabilités dans les bibliothèques tierces
      - Dependency-Check, WhiteSource, Snyk, Black Duck
    - Secret Detection : Traquer les mots de passe et clés API exposés, pour empêcher leur passage dans les commits.
      - GitGuardian, TruffleHog

Etapes du processus CI: analyse de la qualité

- Les 4 rôles fondamentaux de la phase d'analyse:
  - ✓ **Conformité (Compliance) :**
    - Licences : Vérifier que les licences des bibliothèques utilisées sont autorisées par l'entreprise (ex: licence GPL → peut obliger à rendre le code open source)
    - Standards : Contrôler le respect des normes de codage (ISO 27001 : sécurité de l'information, PCI-DSS: sécurité des paiements, etc.)
    - RGPD : impose de protéger les données personnelles
  - ✓ **Gouvernance** (Quality Gates) : un ensemble de critères ou de seuils définis pour évaluer et décider si le code va être bloqué ou autorisé à être deployé.
    - ex: seuil d'acceptabilité : « Le pipeline échoue si la complexité est > 20 ou s'il y a une faille de sécurité majeure ».

Etapes du pipeline CI : build

- La phase du **build** permet de construire l'application exécutable à partir du code source .
- Un ensemble d'actions peuvent être réalisées comme:
  - ✓ Préparation de l'environnement (Setup)
    - Installation des outils nécessaires (Java, Python, Node.js).
    - Nettoyage (Clean) des résidus des builds précédent pour éviter les conflits de fichiers.
  - ✓ Gestion des dépendances
    - Téléchargement des bibliothèques externes (ex: npm install)
    - Vérification SCA : Scan les dépendances pour s'assurer qu'elles ne contiennent pas de vulnérabilités connues avant de les intégrer au produit final.
  - ✓ Compilation du code source: pour les langages compilés,le code est traduit en langage machine

Etapes du processus CI : Tests

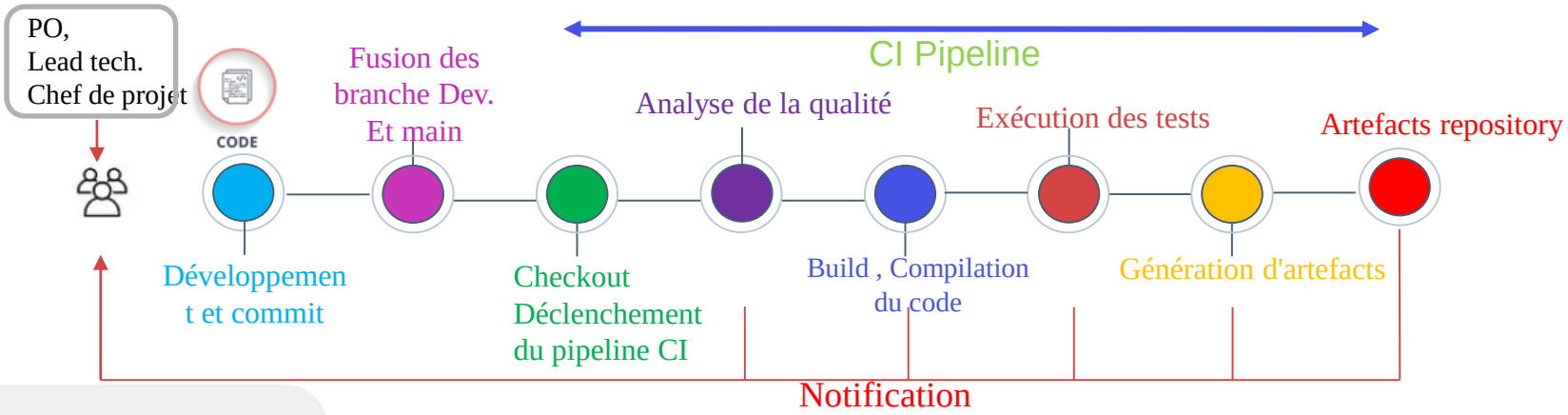
- **Exécution des tests** :
  - ✓ Plusieurs niveaux de tests sont exécutés automatiquement pour garantir la qualité de la nouvelle version.
  - ✓ Ces tests peuvent être des:
    - *Tests Unitaires* : vérifient chaque fonction ou composant individuellement (fonctions, classes).
    - *Tests Intégration* : vérifient l'interaction entre différentes modules du système.
    - *Tests Fonctionnels* : vérifient que les fonctionnalités répondent aux besoins métier .
    - *Tests de régression* : s'assurent que les nouvelles modifications ne constituent pas une régression par rapport à la dernière version.

Etapes du processus CI: artefacts

- Si tous les tests réussissent et que la qualité du code est satisfaisante, on crée des livrables concrets : **artefact**
- Un artefact regroupe le code compilé, les ressources (images, config) et les dépendances dans un fichier unique.
- Il peut prendre différentes formes selon le type d'application et les technologies utilisées :
  - 1) un fichier exécutable: Un fichier JAR, WAR, EXE, etc.
  - 2) une image de Docker
  - 3) un package d'installation: Un RPM, DEB, etc.
  - 4) un bundle de fichiers: un ensemble de fichiers compressés (zip, tar.gz) contenant les ressources de l'application.
- L'artefact est stocké dans un référentiel d'artefacts (Frog Artifactory, Sonatype Nexus) pour être utilisé ultérieurement dans le processus de Livraison continue.

Continue Integration (CI)

- D’autres étapes peuvent être intégrées comme:
  - ✓ la Notification des responsables du pipeline CI, en cas d'échec, ils sont informés des problèmes à résoudre.
  - ✓ L'artefact peut être signé numériquement pour garantir son intégrité et son authenticité.
  - ✓ Ces artefacts vont être utilisés dans les étapes suivantes du processus de **livraison continue**.



Avantages du CI



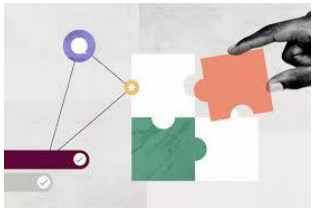
Détection rapide des erreurs : Les problèmes sont identifiés tôt dans le cycle de développement, ce qui réduit les coûts de correction.



Amélioration de la qualité du code grâce aux tests automatisés qui assurent une meilleure qualité.



Augmentation de la fréquence des déploiements permet de livrer de nouvelles fonctionnalités plus rapidement aux utilisateurs.



Meilleure collaboration :
L'intégration continue favorise une collaboration étroite entre les Dévs., Ops, QAs.

Exemple d’outils pour la CI



- **Jenkins**: est un outil open-source qui automatiser les différentes étapes du cycle de vie d'une application, de la compilation à la livraison en passant par les tests (CI/CD).



- **GitLab** : Intégré à GitLab, il offre des fonctionnalités CI/CD.



- **CircleCI**: est une plateforme CI/CD utilisée pour mettre en œuvre des pratiques Devops.



- **Azure DevOps** : offre le service complet CI/CD

C’est quoi Jenkins?



- ✓ Jenkins est une plateforme, écrite en java, open source d'automatisation de l'intégration continue et du déploiement continu.
- ✓ Elle permet aux équipes de développement de construire, tester et déployer automatiquement leur code de manière efficace.

Histoire de Jenkins



Kohsuke Kawaguchi

2004

Hudson: un outil interne pour automatiser les tâches de construction et de test

2010

Conflit entre l'équipe de développement d'Hudson et Oracle

2011

Lancement D'un nouveau projet : **Jenkis**

2016

Dernières mises à jour officielles de **Hudson**

2019

Jenkins X a été accepté par la foundation Cloud Native Computing Foundation (CNCF)

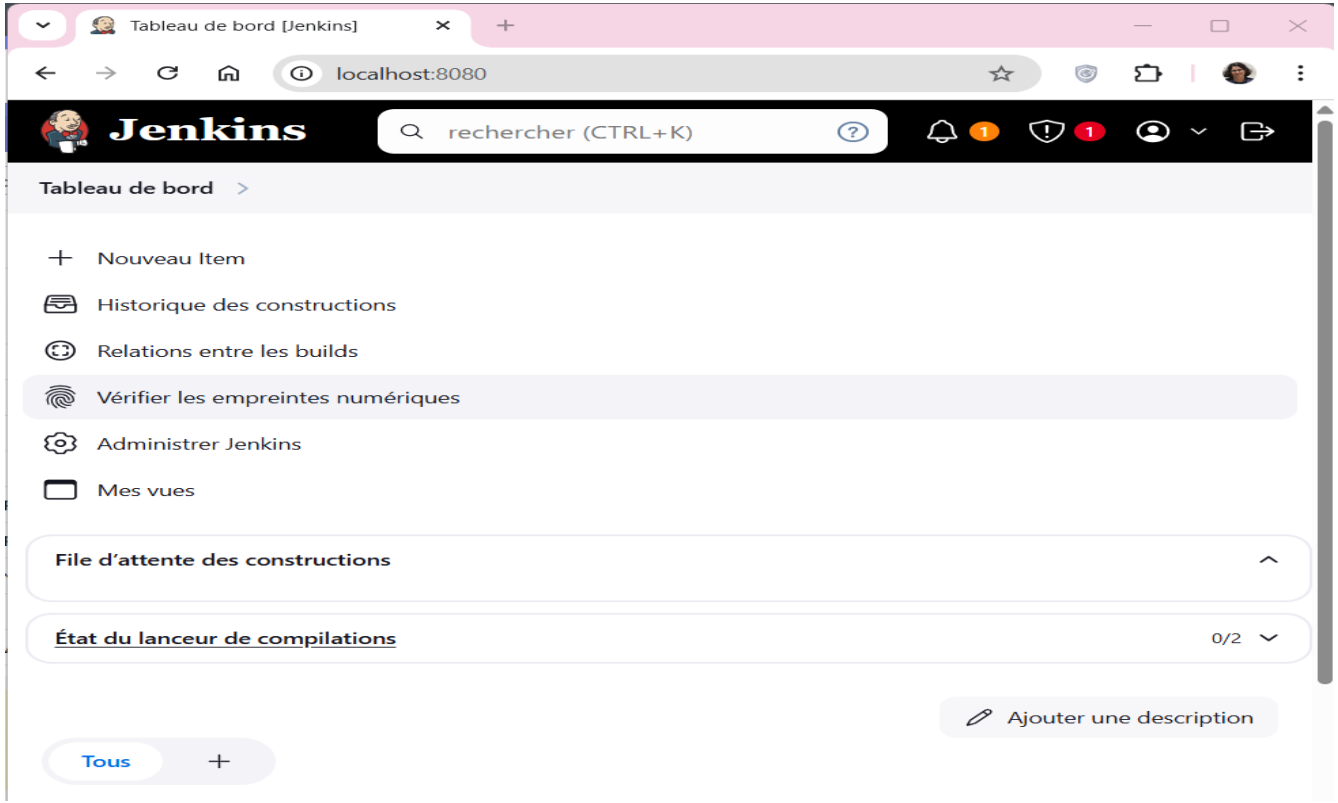
2025

plus de 10 millions de téléchargemepar an Jenkins à travers le monde

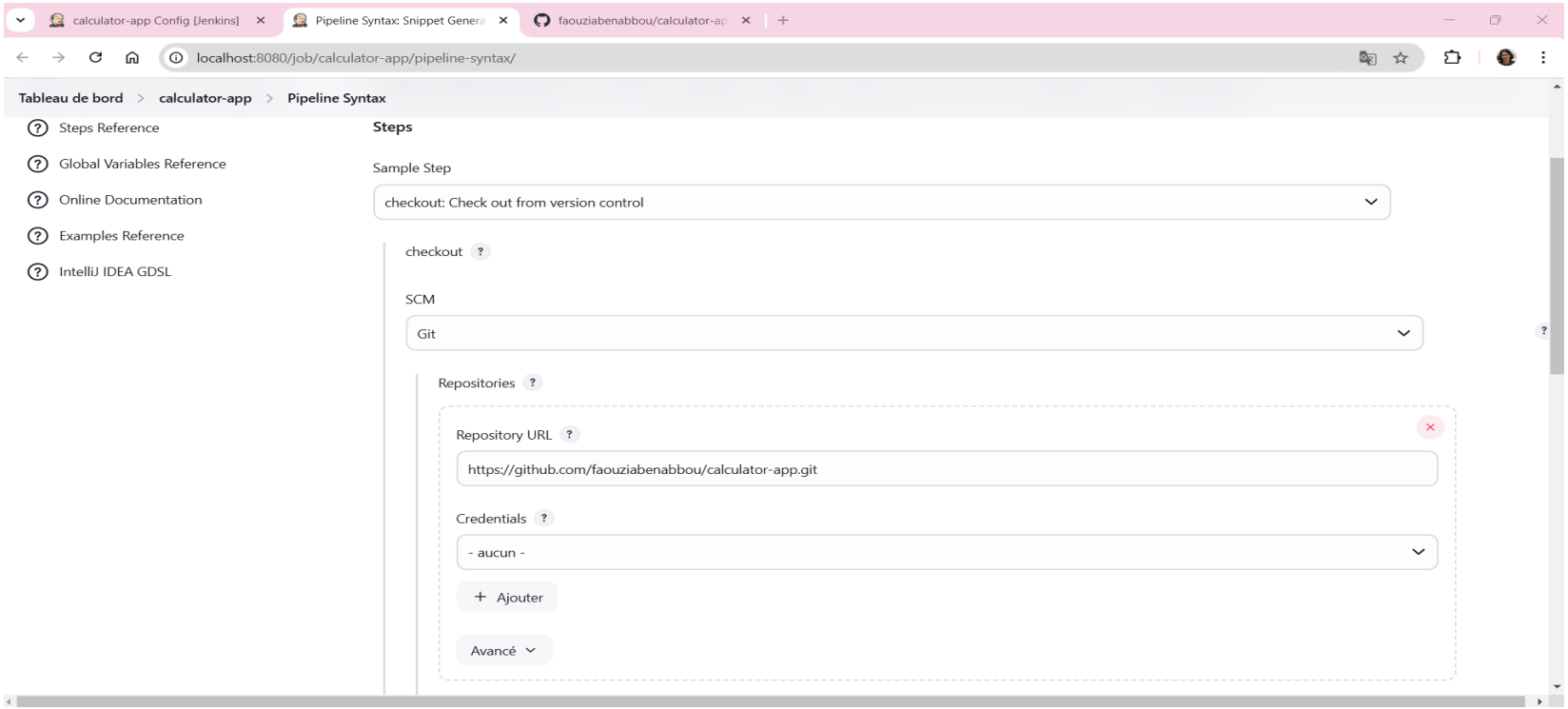
Architecture de Jenkins

- L’architecture de Jenkins comprend deux principaux composants Jenkins Master (ou Controller) et Jenkins Agents (Nodes).
- Jenkins Master
  - Le serveur principal qui orchestre les tâches et gère les interactions utilisateur faites via une interface web, il permet de configurer les objets, de gérer les utilisateurs et les autorisations, et de visualiser l'état des tâches en cours.
  - Gère les plugins pour étendre les fonctionnalités de Jenkins (par exemple, Git, Docker, Kubernetes).
  - Coordonne les agents pour exécuter les tâches.
- Jenkins Agents
  - En environnement local, un agent peut être la même machine physique que le Controller, mais dans des architectures plus avancées, les agents sont souvent séparés (machines physiques ou virtuelles) pour répartir la charge.
  - Les agent exécutent les tâches (job) assignées par le master, telles que la compilation du code, les tests et le déploiement.

Jenkins


Tableau de bord [Jenkins]
localhost:8080
Jenkins
rechercher (CTRL+K)
Tableau de bord
Nouveau Item
Historique des constructions
Relations entre les builds
Vérifier les empreintes numériques
Administrer Jenkins
Mes vues
File d'attente des constructions
État du lanceur de compilations
0/2
Ajouter une description
Tous

Jenkins


calculator-app Config [Jenkins]
Pipeline Syntax: Snippet Genera
faouziabenabbou/calculator-ap
localhost:8080/job/calculator-app/pipeline-syntax/
Tableau de bord > calculator-app > Pipeline Syntax
Steps Reference
Global Variables Reference
Online Documentation
Examples Reference
IntelliJ IDEA GDSL
Steps
Sample Step
checkout: Check out from version control
checkout
SCM
Git
Repositories
Repository URL
https://github.com/faouziabenabbou/calculator-app.git
Credentials
- aucun -
Ajouter
Avancé

Jenkins



main · 2 Branches · 0 Tags · Go to file t · Add file · Code

faouziabenabbou Update index.js — b16d478 · 20 minutes ago — 35 Commits

test	Update app.test.js.txt add stop server	29 minutes ago
README.md	Update README.md	last year
index.js	Update index.js	20 minutes ago
index.test.js	Create index.test.js	last year
jenkinsfile	Update jenkinsfile missing }	1 hour ago
package.json	add depency express	44 minutes ago
server.js	Create server.js	22 minutes ago

README

calculator-app

pipeline jenkins test

Jenkins

- **Jenkinsfile**
  - C'est un fichier texte écrit en **Groovy**
  - Un Jenkinsfile est généralement structuré en plusieurs étapes.
  - Chaque étape représente une phase du pipeline CI/CD, comme la compilation, les tests, le déploiement, etc.
  - Ces étapes seront automatiquement exécutées.
  - Exemple
    - ✓ **Checkout** : récupérer le code depuis Git
    - ✓ **Installer les dépendances** : npm install
    - ✓ **Lancer les tests** : npm test
    - ✓ **Build** / packaging : préparer l'application pour la production
    - ✓ **Débogage**: vérifier le workspace, versions Node, npm, ou les fichiers générés

```
pipeline {
    agent any
    tools {
        nodejs "NodeJS" // Nom de l'installation configurée dans Jenkins
    }
    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }
        stage('Install Dependencies') {
            steps {
                powershell 'npm install'
            }
        }
        stage('Run Tests') {
            steps {
               // powershell 'echo "Skipping tests for now"'
                          powershell 'npm test'
            }
        }
        stage('Build') {
            steps {
                powershell 'echo "Build step completed!"'
            }
        }
        stage('Debug') {
            steps {
                powershell 'dir'
            }
        }
    }
}
```

Jenkins



- **Package.json**
  - Il sert à:
    - ✓ Décrire le projet: Nom, version, auteur, licence, description, type de licence (ISC Internet Systems Consortium)
    - ✓ Point d'entrée de l'application (fichier principal)
    - ✓ Scripts que l'on peut lancer avec **npm run;** ici Lance le serveur Node.js et les tests avec Jest
    - ✓ Dépendances nécessaires uniquement pour le développement : Jest et supertest
    - ✓ Configuration spécifique pour Jest: indique que les tests s'exécutent dans un environnement Node.
    - ✓ Dépendances nécessaires pour exécuter l'application en production

```
{
  "name": "calculator-app",
  "version": "1.0.0",
  "description": "pipeline jenkins test",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "test": "jest"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "devDependencies": {
    "jest": "^29.7.0",
    "supertest": "^6.3.3"
  },
  "jest": {
    "testEnvironment": "node"
  },
  "dependencies": {
    "express": "^5.2.1"
  }
}
```

https://jsonlint.com/

Jenkins



- Exemple de Pipeline

Graph [calculator-app #46] [Jen

localhost:8080/job/calculator-app/46/pipeline-graph/

Jenkins | rechercher (CTRL+K) | faouzia benabbou | se déconnecter

Tableau de bord > calculator-app > #46 > Pipeline Overview

Build #46

Rebuild | Console | Configure

Pipeline

Start | *Checkout SCM* | *Tool Install* | Checkout | Install Depende... | Run Tests | Build | Debug | End

Details

- Manually run by faouzia benabbou
- Started 1 h 24 mn ago
- Queued 4 ms
- Took 30 s

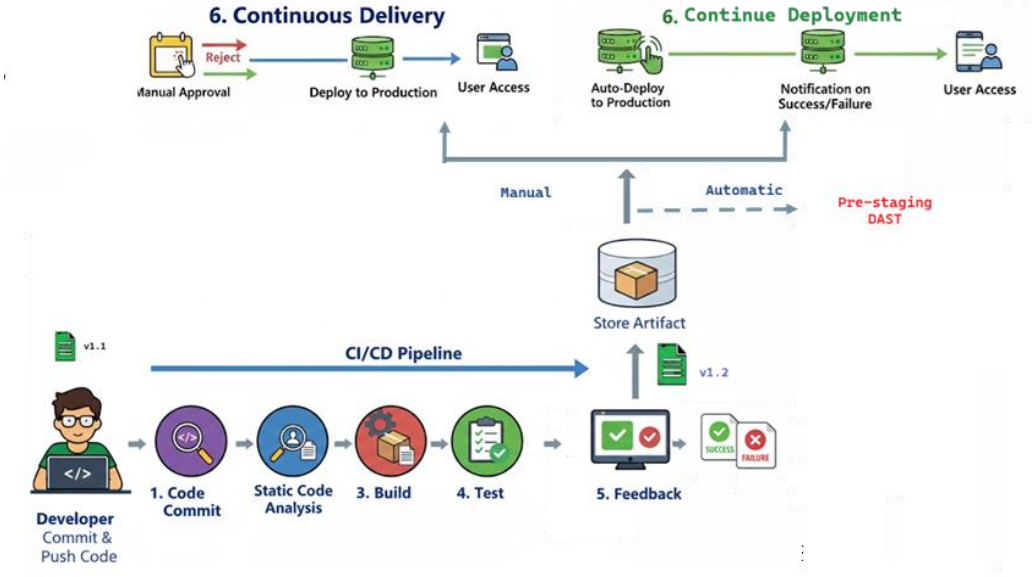
Jenkins



- Pour Automatiser, on peut configurer un **webhook** : c'est un mécanisme qui déclenche une action sur un serveur lorsqu'un évènement se produit ailleurs
- Dans notre cas on veut que GitHub déclenche le build directement sur le serveur Jenkins
- GitHub doit pouvoir atteindre votre serveur local depuis Internet
- **Ngrok** crée un tunnel HTTPS vers le serveur local, pour qu'il soit accessible depuis Internet.

la Livraison / déploiement Continue

- L'intégration continue permet de construire des artefacts qu'on stocke dans des dépôts.
- Le CD est responsable du **déploiement** des modifications du code dans l'environnement de **production**.



la Livraison / déploiement Continue

- **Les étapes du CD**
  - Pré-production (ou pre-Staging)
  - Validation manuelle ou Automatique
  - Déploiement en production selon une stratégie

Continue Deployment

- Pré-production
  - *Tests supplémentaires*
    - ✓ Dynamic Application Security Testing (DAST) permet de tester les vulnérabilités de sécurité sur une application en cours d'exécution.
    - ✓ Tests d'intégration permet de vérifier que l'application communique correctement avec les bases de données et services externes.
    - ✓ Tests techniques avancés comme les tests de performance, de charge et de stabilité.
- Si tous les tests passent avec succès l'application est automatiquement déployé en production.

Continue Delivery

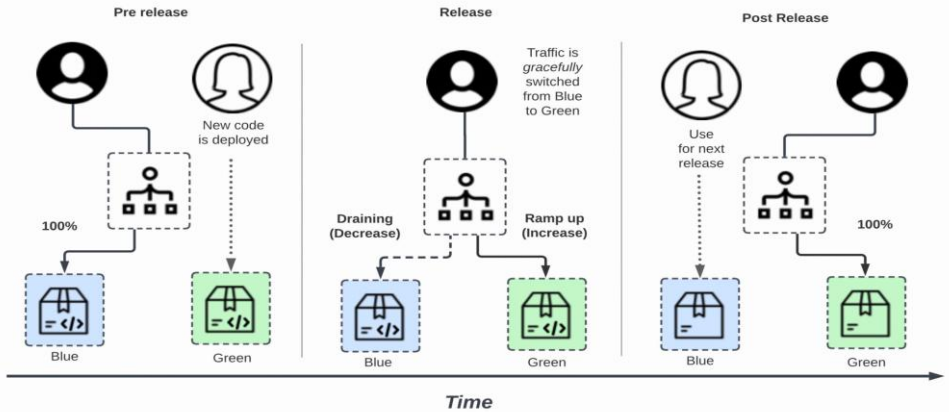
- Pré-production
  - *Tests manuels*: Des testeurs peuvent effectuer des tests manuels pour vérifier des scénarios complexes ou des fonctionnalités qui sont difficiles à automatiser.
- Approbation: Les responsables donnent leur approbation pour le déploiement en production après avoir vérifié que le logiciel est prêt.
- Si l'approbation est donnée, l'application est déployé en production.

Déploiement en production

- *Stratégies de déploiement*
  - Il existe différentes stratégies de déploiement
    - ✓ le déploiement **bleu-vert**,
    - ✓ le déploiement **canari**,
    - ✓ le déploiement **progressif**.

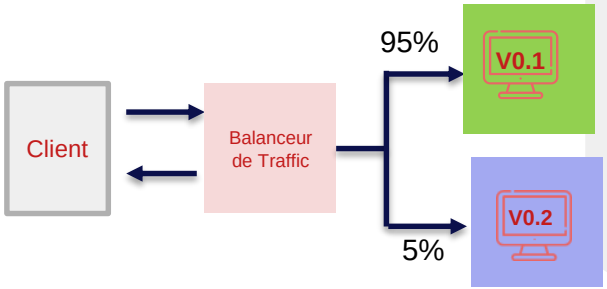
Stratégies de déploiement

- Le déploiement **bleu-vert**
  - ✓ On maintient deux environnements de production strictement identiques et actifs, mais un seul reçoit le trafic public.
    - Environnement Bleu : Version stable actuelle (v1.0), Active et Public.
    - Environnement Vert : Nouvelle version (v1.1), Active mais Privé.
  - ✓ Une fois la nouvelle version validée, le trafic est basculé de l'environnement bleu vers l'environnement vert de manière instantanée.
    - Le Vert devient le nouvel environnement Public
    - Le Bleu devient l'environnement de Privé
  - ✓ Si un problème survient, on rebascule le trafic vers l'environnement bleu.



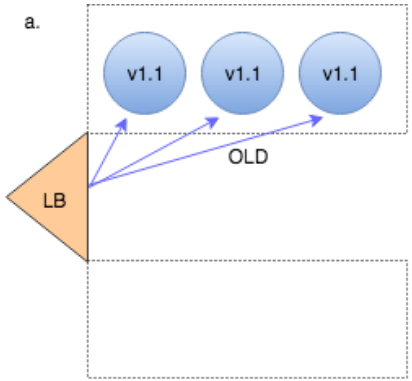
Stratégies de déploiement

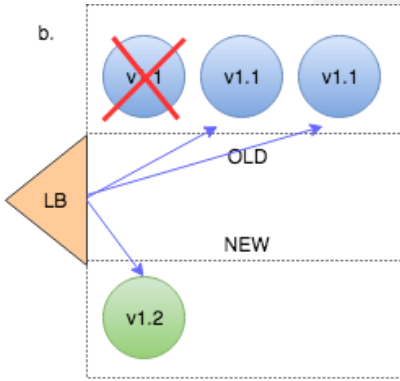
- Le déploiement **canari**
  - ✓ Il s'agit de tester la nouvelle version avec un petit groupe d'utilisateurs (les "canaris").
    - Exemple 1 serveur en V1.2, 100 serveurs en v1.1 et 4% des utilisateurs utilisent la version1.2.
  - ✓ Le comportement de la nouvelle version est étroitement surveillé pour détecter d'éventuels problèmes.
  - ✓ Si aucun problème n'est détecté, la nouvelle version est déployée à un groupe d'utilisateurs plus large, etc.
  - ✓ Le déploiement canari est une approche plus spécifique qui se concentre sur la **détection rapide des problèmes**

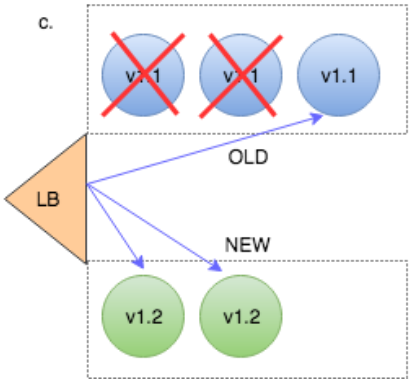


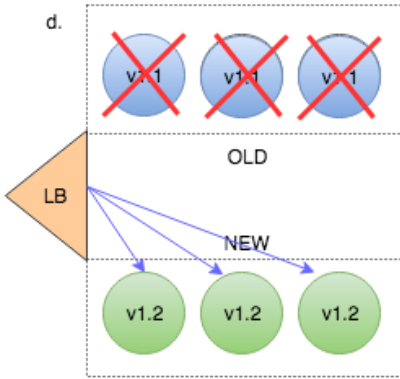
Stratégies de déploiement

- Le déploiement **progressif (rolling deployment)**
  - ✓ On remplace progressivement la version 1.1 par la nouvelle version sur les serveurs progressivement .
  - ✓ On a plusieurs instances (serveurs, containers, pods) derrière un load balancerExemple : 4 ou 5 serveurs
  - ✓ On met à jour les serveurs un par un ou par petits groupes
  - ✓ Le loadbalancer Réparti le trafic proportionnellement au nombre de serveurs mis à jour (ex: si 1 serveur sur 4 est mis à jour, 25% des utilisateurs auront la V1.2 automatiquement).





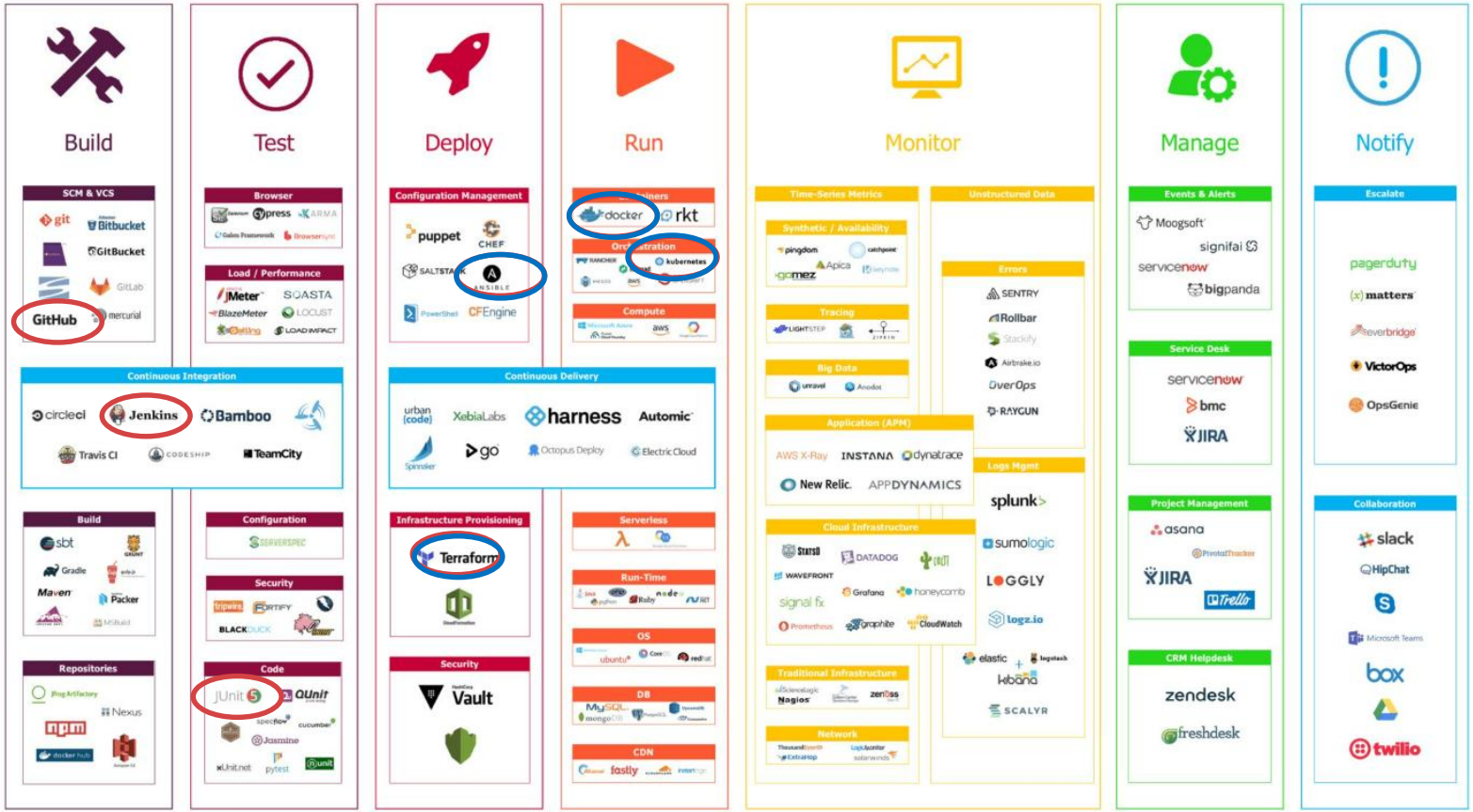




Stratégies de déploiement

Critère	Bleu-Vert (Blue-Green)	Canari (Canary)	Progressif (Rolling)
Objectif principal	Assurer un déploiement sans interruption avec rollback immédiat	Réduire le risque en testant sur un petit groupe d'utilisateurs	Mettre à jour progressivement tous les serveurs sans arrêt de service
Principe	Deux environnements : ancien (Blue) et nouveau (Green), puis on bascule	Nouvelle version déployée à un faible taux d'utilisateurs puis augmentation progressive	Remplacement progressif des instances une par une
Gestion du trafic	Basculement total : 100% d'un coup	Contrôle qui des utilisateurs est aiguillé vers la nouvelle version : ex: 5% puis 25% puis 100%	Répartition automatique selon les serveurs disponibles
Infrastructure	Double : 2 environnements complets	Infrastructure standard + gestion du routage du trafic	Infrastructure non dupliquée
Complexité	Moyenne	Élevée car ca nécessite le monitoring et routage avancé	Faible
Rollback (retour arrière)	Très rapide : on rebascule vers Blue	Rapide, on coupe le trafic vers la nouvelle version	Plus lent : rollback serveur par serveur

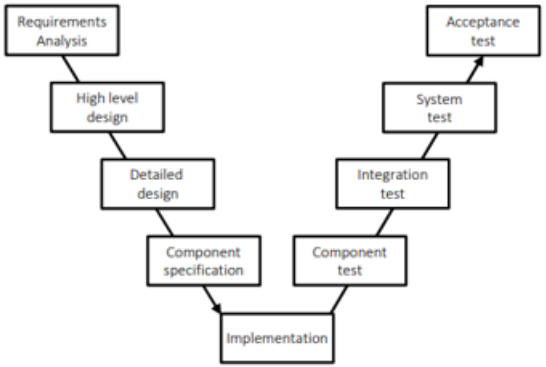
Outils populaires pour CI/CD



Test Continue

Tests Logiciels

- Dans les approches non agiles, les tests constituent la dernière phase du cycle de vie de développement Logiciel.
- Si un bug était détecté lors de la phase de test, il était difficile et coûteux de revenir en arrière et d'apporter les modifications nécessaires.
- Les testeurs de logiciels faisaient partie d'une équipe distincte, isolée de l'équipe de développement.



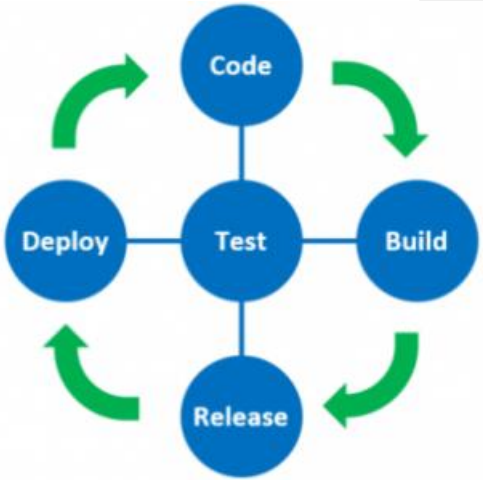
Le modèle en V de Boehm B.

L'approche de test « Shift Left »

- La culture DevOps adopte l'approche de test « Shift Left », qui contraste avec les environnements traditionnels où les tests ont lieu à la fin du développement.
- Cette approche pousse les tests vers la gauche, c'est-à-dire vers les premières étapes du processus de développement logiciel.
- Là aussi, les tests sont lancés dès le début du développement.
- Cette approche permet d'identifier les bugs le plus tôt possible et contribue de manière significative à l'amélioration de la qualité des logiciels.

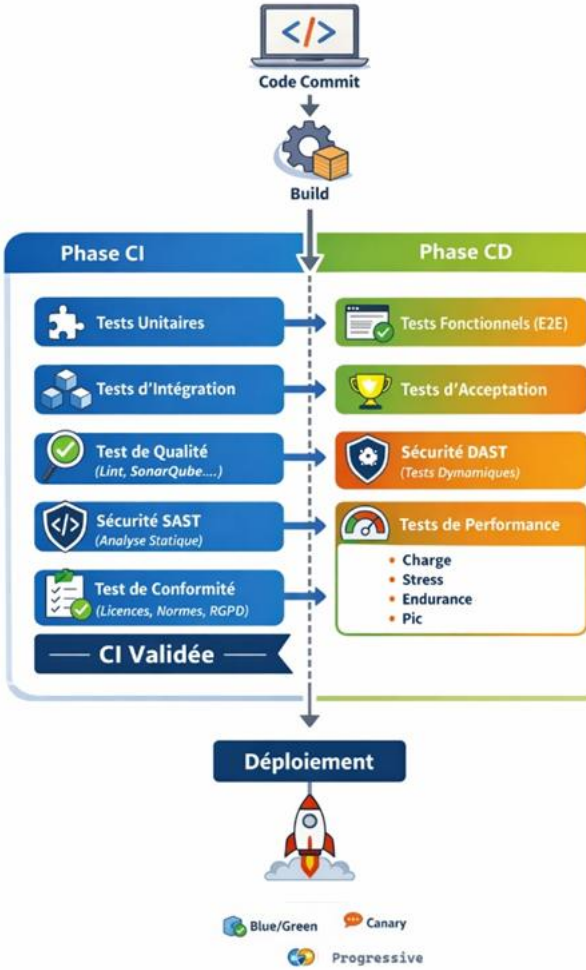
Test Automtisés dans CI/CD

- les tests automatisés jouent un rôle crucial dans CI/CD, ils permettent de vérifier de manière systématique et répétée la qualité du code à chaque modification, garantissant ainsi une livraison continue et fiable.
- Les tests couvrent chaque étape du cycle de vie du développement logiciel ce qui minimise les retours en arrière dans le cas où une erreur est détectée.
- De plus, les tests est une responsabilité partagée entre l'ensemble de l'équipe ce qui permet à chacun de comprendre l'impact de chaque modification dans le code.



Types de Tests Logiciels

- Ces tests couvrent différents niveaux :
  - **Tests unitaires**
  - **Tests d'intégration**
  - **Test de qualité**
  - **Test de sécurité SAST** (Static Application Security Testin et DAST (Dynamic Application Security Testing)
  - **Test de conformité**
  - **Tests fonctionnels**
  - **Test de sécurité** et DAST (Dynamic Application Securi Testing)
  - **Tests d'acceptation UAT**.
  - **Tests de performance**

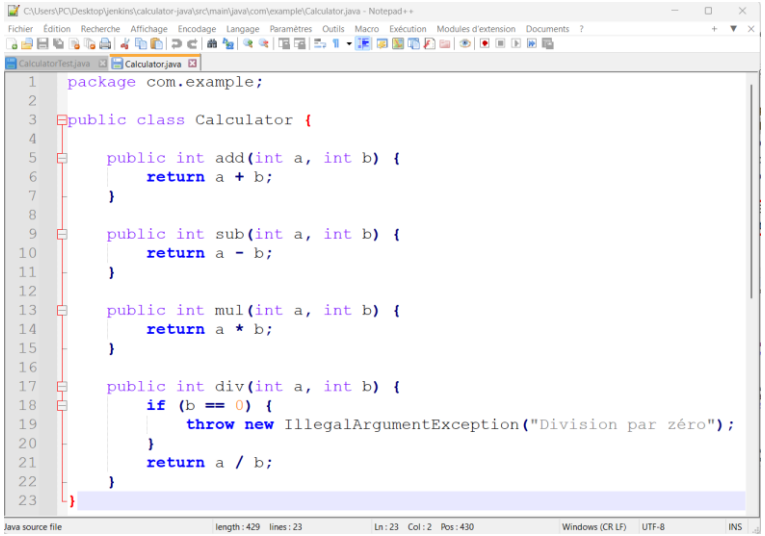


Outils d'automatisation des tests

	Type de test	Outil	Plateformes supportées	Licence
Selenium	Tests automatisés web	Java, Python, C#, JavaScript	Web	Open-source
Katalon Studio	Tests automatisés web, API, desktop, mobile	Groovy, Java	Web, API, Desktop, Mobile	Gratuit / Payant
JMeter	Tests de performance & charge	Java	Web, API	Open-source
SoapUI	Tests API (REST, SOAP, GraphQL)	Java	API	Open-source
Unit	Tests unitaires	Java	Applications Java	Open-source
Jest	Tests unitaires et fonctionnels	JavaScript	Web, Node.js	Open-source

Exemple complet avec JUNIT

- Pipeline CI/CD complet avec des tests
- Projet simple Calculator-java
- Test avec JUnit



```java
package com.example;

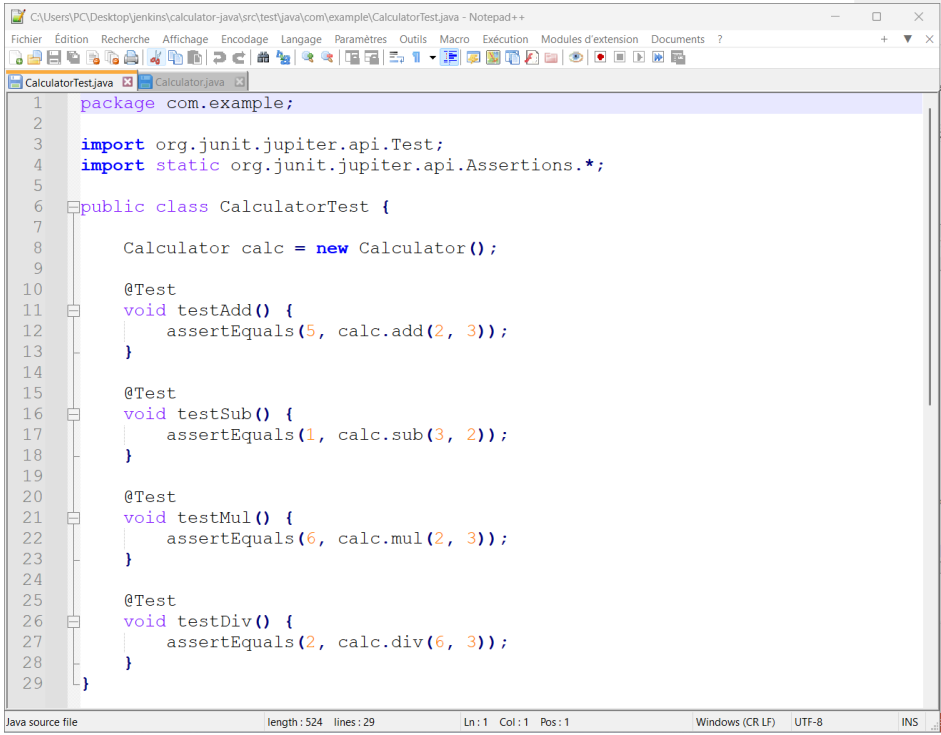
public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }

    public int sub(int a, int b) {
        return a - b;
    }

    public int mul(int a, int b) {
        return a * b;
    }

    public int div(int a, int b) {
        if (b == 0) {
            throw new IllegalArgumentException("Division par zéro");
        }
        return a / b;
    }
}
```



```java
package com.example;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class CalculatorTest {

    Calculator calc = new Calculator();

    @Test
    void testAdd() {
        assertEquals(5, calc.add(2, 3));
    }

    @Test
    void testSub() {
        assertEquals(1, calc.sub(3, 2));
    }

    @Test
    void testMul() {
        assertEquals(6, calc.mul(2, 3));
    }

    @Test
    void testDiv() {
        assertEquals(2, calc.div(6, 3));
    }
}
```

Exemple complet

- Jenkinsfile et package.json

C:\Users\PC\Desktop\jenkins\calculator-java\packa...

Fichier Édition Recherche Affichage Encodage Langage Paramètres
Outils Macro Exécution Modules d'extension Documents ? + ▼

Jenkinsfile | package.json

```
1  {
2    "name": "calculator-app",
3    "version": "1.0.0",
4    "scripts": {
5      "build": "mvn clean compile",
6      "test": "mvn test"
7    }
8  }
```

Ln : 1 Col : 1 Pos : 1 Windows (CR LF) UTF-8 INS

Jenkinsfile

```
1   pipeline {
2       agent any
3
4       tools {
5           maven 'Maven 3'
6           jdk 'JDK 21'
7       }
8
9       stages {
10          stage('Git Test') {
11              steps {
12                  git branch: 'main',
13                      url: 'https://github.com/faouziabenabbou/calculator-java.git'
14              }
15          }
16          stage('Test SCM') {
17              steps {
18                  echo "SCM OK"
19                  powershell 'Get-ChildItem -Recurse'
20              }
21          }
22          stage('Checkout') {
23              steps {
24                  echo "Récupération du code depuis Git..."
25                  checkout scm
26              }
27          }
28          stage('Build') {
29              steps {
30                  echo 'Compilation avec Maven'
31                  powershell 'mvn clean compile'
32              }
33          }
34          stage('Test') {
35              steps {
36                  echo 'Exécution des tests unitaires JUnit'
37                  powershell 'mvn test'
38              }
39          }
40          stage('Package') {
41              steps {
42                  echo 'Packaging de l'application'
43                  powershell 'mvn package'
44              }
45          }
46          stage('Archive Artifact') {
47              steps {
48                  echo 'Archivage du JAR'
49                  archiveArtifacts artifacts: 'target/*.jar', fingerprint: true
50              }
51          }
52
```

length : 1 370 li Ln : 1 Col : 1 Pos : 1 Windows (CR LF) UTF-8 INS

ent sélectionné 1,33 Ko

Exemple complet av

- Déroulement du Pipeline
- Aperçue d’une partie de l’exécution du pipeline

Pipeline




```
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ calculator-java ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory C:\Users\PC\AppData\Local\Jenkins\.jenkins\workspace\Calculator-java\src\main\resources
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ calculator-java ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ calculator-java ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory C:\Users\PC\AppData\Local\Jenkins\.jenkins\workspace\Calculator-java\src\test\resources
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ calculator-java ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- surefire:3.1.2:test (default-test) @ calculator-java ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.example.CalculatorTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.089 s -- in com.example.CalculatorTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.4.1:jar (default-jar) @ calculator-java ---
[INFO] Building jar: C:\Users\PC\AppData\Local\Jenkins\.jenkins\workspace\Calculator-java\target\calculator-java-1.0-SNAPSHOT.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  4.479 s
[INFO] Finished at: 2026-02-27T00:24:29Z
[INFO] ------------------------------------------------------------------------
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Archive Artifact)
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] withEnv
[Pipeline] {
[Pipeline] echo
Archivage du JAR
[Pipeline] archiveArtifacts
Archiving artifacts
Enregistrement des empreintes numériques
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Avantages des tests automatisés dans CI/CD

- **Gain de temps**: Les tests sont exécutés automatiquement, ce qui accélère le processus de validation du code.
- **Gain d'efforts**: Une fois les scripts de test créés, ils peuvent être réutilisés plusieurs fois sans effort supplémentaire, réduisant considérablement la charge de travail.
- **Augmentation de la fiabilité**: Les tests automatisés sont moins sujets aux erreurs humaines, garantissant des résultats cohérents et fiables.
- **Détection rapide des bugs**: Les bugs sont identifiés dès les premières étapes du développement, réduisant les coûts de correction.
- **Élargissement de la couverture des tests** : Ils permettent de tester un plus grand nombre de cas et scénarios.
- **Amélioration de la qualité Logiciel**: En détectant et corrigeant les problèmes plus tôt, les tests automatisés contribuent à améliorer la stabilité et la qualité globale des applications.
- **Support multi-environnements** : On peut tester facilement le code sur différents environnements, systèmes d'exploitation ou configurations, augmentant la portabilité et la compatibilité.